

EcoTwin System Design Document (SDD)

Architecture
Technical Design Specification

June 12, 2026

Contents

1	Introduction	4
2	Architecture Overview	4
2.1	High-Level Architecture	4
3	Technology Stack	5
3.1	Frontend	5
3.2	Backend	5
3.3	Database	5
3.4	ML Layer	5
3.5	Remote Sensing	6
4	Region Architecture	6
4.1	Region Definition	6
4.2	Reasoning	6
4.3	Region Structure	6
5	Data Pipeline	6
5.1	Step 1: Generate Grid	6
5.2	Step 2: Sentinel Retrieval	7
5.3	Step 3: Cloud Masking	7
5.4	Step 4: Composite Generation	7
6	Feature Extraction	7
6.1	Environmental Indicators	7
6.1.1	NDVI	7
6.1.2	NDWI	7
6.1.3	NBR	7
6.2	Stored Statistics	7
7	Embedding Pipeline	8
7.1	Input	8
7.2	Model	8
7.3	Output	8
7.4	Storage	8

8	Database Design	8
8.1	regions	8
8.2	region <i>features</i>	8
8.3	region <i>embeddings</i>	9
8.4	temporal <i>profiles</i>	9
8.5	reports	9
9	Vector Search Design	10
9.1	Search Workflow	10
9.2	Similarity Metric	10
9.3	Default	10
10	Forecasting Engine	10
10.1	Forecasting Philosophy	10
10.2	Workflow	10
10.3	Forecast Inputs	11
10.4	Forecast Outputs	11
11	API Architecture	11
11.1	Region APIs	11
11.1.1	Get Region	11
11.1.2	Find Region	11
11.2	Similarity APIs	12
11.2.1	Search Analogs	12
11.3	Temporal APIs	12
11.3.1	Temporal Profile	12
11.4	Forecast APIs	12
11.4.1	Forecast	12
11.5	Report APIs	13
11.5.1	Generate Report	13
12	Frontend Design	13
12.1	Map Page	13
12.2	Region Panel	13
12.3	Analog Panel	13
12.4	Forecast Panel	13
13	Caching Strategy	14
13.1	Redis	14
14	Security	14
14.1	Authentication	14
14.2	Authorization	14
14.3	Rate Limiting	14
15	Deployment Architecture	14
15.1	Containers	14
15.2	Docker Compose	14
16	Cloud Deployment	15
16.1	Infrastructure	15
16.2	Production Stack	15

17 Monitoring	15
18 Performance Targets	15
19 Future Enhancements	16

1 Introduction

This document describes the technical architecture, system components, database design, data flow, APIs, deployment strategy, and implementation details for EcoTwin.

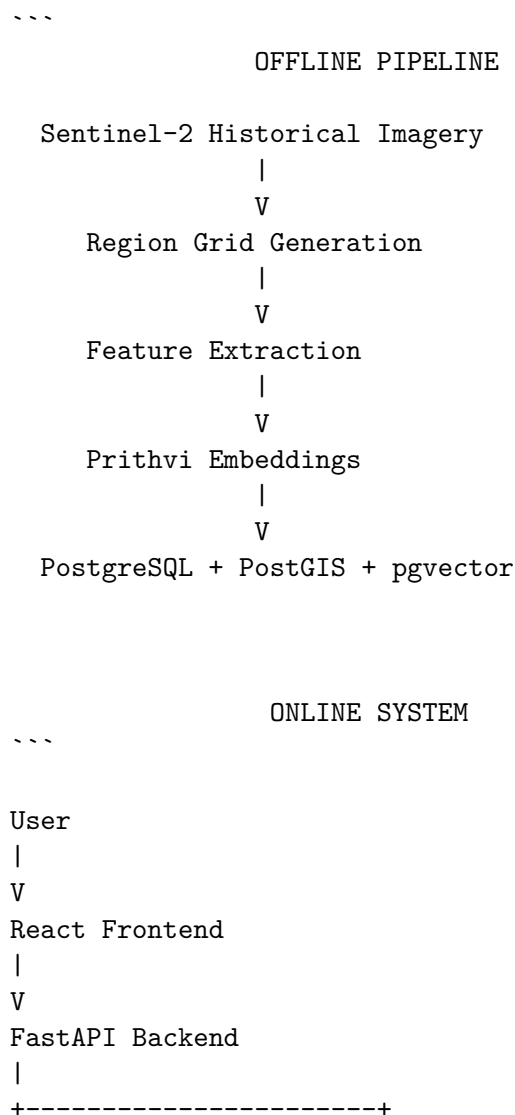
EcoTwin is a geospatial ecosystem discovery and analog forecasting platform.

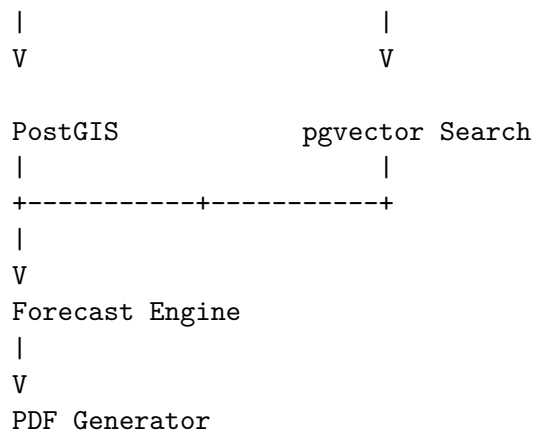
The platform enables:

- Ecosystem discovery
- Similarity search
- Temporal ecosystem analysis
- Analog-based forecasting
- Automated report generation

2 Architecture Overview

2.1 High-Level Architecture





3 Technology Stack

3.1 Frontend

- React
- TypeScript
- Vite
- React Leaflet
- Plotly
- TailwindCSS

3.2 Backend

- FastAPI
- SQLAlchemy
- Alembic
- Pydantic

3.3 Database

- PostgreSQL
- PostGIS
- pgvector

3.4 ML Layer

- PyTorch
- Prithvi Foundation Model
- NumPy
- Pandas

3.5 Remote Sensing

- Google Earth Engine
- Sentinel-2 SR Harmonized
- Sentinel Cloud Probability

4 Region Architecture

4.1 Region Definition

A region represents the smallest searchable ecosystem unit.

Region size:

[5km $\times$ 5km]

Area:

[25km²]

4.2 Reasoning

Using a fixed grid provides:

- Fast indexing
- Uniform ecosystem representation
- Easy retrieval
- Simple forecasting

4.3 Region Structure

Region ID

Center Latitude

Center Longitude

Bounding Box

Polygon Geometry

Area

5 Data Pipeline

5.1 Step 1: Generate Grid

Input:

- Project Boundary

Output:

- Region Grid

5.2 Step 2: Sentinel Retrieval

Dataset:

COPERNICUS/S2_SR_HARMONIZED

Period:

2018-2025

5.3 Step 3: Cloud Masking

Apply:

- SCL Mask
- Cloud Probability Mask

5.4 Step 4: Composite Generation

Method:

Median Composite

Output:

- RGB Composite
- Spectral Composite

6 Feature Extraction

6.1 Environmental Indicators

6.1.1 NDVI

$$[\text{NDVI} = \text{B8-B4}_{\overline{\text{B8+B4}}}]$$

6.1.2 NDWI

$$[\text{NDWI} = \text{B3-B8}_{\overline{\text{B3+B8}}}]$$

6.1.3 NBR

$$[\text{NBR} = \text{B8-B12}_{\overline{\text{B8+B12}}}]$$

6.2 Stored Statistics

For every region:

- Mean
- Median
- Standard Deviation
- Minimum
- Maximum

7 Embedding Pipeline

7.1 Input

Sentinel-2 Composite

Bands:

B2

B3

B4

B8

B11

B12

7.2 Model

Prithvi Foundation Model

7.3 Output

Embedding:

768 Dimensions

7.4 Storage

Stored in pgvector.

8 Database Design

8.1 regions

```
CREATE TABLE regions (  
  
  region_id UUID PRIMARY KEY,  
  
  center_lat DOUBLE PRECISION,  
  
  center_lon DOUBLE PRECISION,  
  
  area_km DOUBLE PRECISION,  
  
  geom GEOMETRY(POLYGON,4326),  
  
  created_at TIMESTAMP  
  
);
```

8.2 region_features

```
CREATE TABLE region_features (  
  
  region_id UUID,
```

```

year INTEGER,

ndvi_mean REAL,
ndvi_std REAL,

ndwi_mean REAL,
ndwi_std REAL,

nbr_mean REAL,
nbr_std REAL,

dominant_ecosystem TEXT

);

```

8.3 *region_embeddings*

```

CREATE TABLE region_embeddings (

region_id UUID,

year INTEGER,

embedding VECTOR(768)

);

```

8.4 *temporal_profiles*

```

CREATE TABLE temporal_profiles (

region_id UUID,

year INTEGER,

ndvi REAL,

ndwi REAL,

nbr REAL

);

```

8.5 *reports*

```

CREATE TABLE reports (

report_id UUID,

region_id UUID,

generated_at TIMESTAMP,

```

```
pdf_url TEXT
```

```
);
```

9 Vector Search Design

9.1 Search Workflow

```
Selected Region
```

```
|  
V
```

```
Retrieve Embedding
```

```
|  
V
```

```
pgvector Similarity Search
```

```
|  
V
```

```
Top K Analog Regions
```

9.2 Similarity Metric

Cosine Similarity

9.3 Default

[K = 10]

10 Forecasting Engine

10.1 Forecasting Philosophy

Forecasting uses ecosystem analogs.

No direct predictive model is used.

10.2 Workflow

```
Current Region
```

```
|  
V
```

```
Find Historical Analog
```

```
...  
|  
V
```

...

Observe Future Evolution

...

|
V
...

Generate Forecast

10.3 Forecast Inputs

- Current Embedding
- Historical Embeddings
- NDVI History
- NDWI History
- NBR History

10.4 Forecast Outputs

- Vegetation Trend
- Water Trend
- Burn Severity Trend
- Confidence Score

11 API Architecture

11.1 Region APIs

11.1.1 Get Region

GET

/api/v1/regions/{id}

Response:

Region Metadata

11.1.2 Find Region

POST

/api/v1/regions/query

Request:

```
{  
lat,  
lon  
}
```

Response:

```
{  
region_id  
}
```

11.2 Similarity APIs

11.2.1 Search Analogs

GET

/api/v1/similarity/{region_id}

Response:

```
[  
analog_1,  
analog_2,  
analog_3  
]
```

11.3 Temporal APIs

11.3.1 Temporal Profile

GET

/api/v1/temporal/{region_id}

11.4 Forecast APIs

11.4.1 Forecast

GET

/api/v1/forecast/{region_id}

Response:

```
{  
ndvi_forecast,  
ndwi_forecast,  
confidence  
}
```

11.5 Report APIs

11.5.1 Generate Report

POST

/api/v1/report

Response:

PDF URL

12 Frontend Design

12.1 Map Page

Features:

- Satellite Basemap
- Region Selection
- Analog Visualization

12.2 Region Panel

Displays:

- Region ID
- Ecosystem Type
- NDVI
- NDWI
- NBR

12.3 Analog Panel

Displays:

- Similarity Score
- Region Metadata
- Location

12.4 Forecast Panel

Displays:

- Forecast Horizon
- Predicted Trends
- Confidence

13 Caching Strategy

13.1 Redis

Cache:

- Region Metadata
- Similarity Results
- Forecast Results

TTL:

24 Hours

14 Security

14.1 Authentication

JWT Tokens

14.2 Authorization

Role Based Access

14.3 Rate Limiting

Per User Request Limits

15 Deployment Architecture

15.1 Containers

Frontend

Backend

PostgreSQL

Redis

15.2 Docker Compose

Services:

- frontend
- backend
- postgres
- redis

16 Cloud Deployment

16.1 Infrastructure

Recommended:

- DigitalOcean
- AWS
- Azure
- GCP

16.2 Production Stack

Nginx

Frontend Container

Backend Container

PostgreSQL

Redis

Object Storage

17 Monitoring

Tools:

- Prometheus
- Grafana
- Loki

Metrics:

- API Latency
- Search Latency
- Forecast Latency
- Error Rates

18 Performance Targets

Operation	Target
	Region Lookup
<100ms	<500ms
Similarity Search	Forecast
<2 sec	<10 sec
Report Generation	

19 Future Enhancements

- Dynamic Ecosystem Boundaries
- Global Search
- Climate Data Integration
- Multi-Model Forecasting
- Mobile Application
- Real-Time Monitoring